Contents

Introduction **3**

1 Literature Review **6**

1.1 Computational humor as a research problem 6
1.2 Evolution of approaches to humor generation 7
1.3 The issues of humor evaluation 8
1.4 The promise of reasoning for humor generation 9
1.5 Standard post-training is a poor fit for humor 10
1.6 Reasoning-oriented post-training 11
1.7 Research gap 12

2 Methodology **13**

2.1 Research scope and implementation status 13
2.2 Dataset construction 13
2.3 Dense embedding index 14
2.4 Keyword extraction as weak prompt retrieval 14
2.5 Reference construction by keyword grouping and retrieval 16
2.5.1 Keyword grouping and prompt rendering 16
2.5.2 Reference retrieval with embedding index 16
2.5.3 Deduplication and splitting 18
2.6 Candidate generation 18
2.7 Candidate evaluation 19
2.7.1 Pairwise comparison 19
2.7.2 Leaderboard aggregation 19
2.8 Theoretical framework for MRVF training 20
2.8.1 Notation 20
2.8.2 RL with verifiable rewards 21
2.8.3 VeriFree 23

2.8.4 Multi-reference VeriFree 24
2.8.5 Incomplete observed references 27
2.8.6 Proposed training loop 29
2.8.7 Note on baselining 31

3 Results **33**
3.1 Current results 33
3.2 Planned training procedure 33
3.3 Planned ablations 34

References **36**

Introduction

Humor generation is a useful way to study the limits of current language-model post-training. In domains such as mathematics or code, a generated answer can often be checked by an external procedure: a solution is either equal to the correct answer, or a program either passes the tests or fails them. Humor does not have that structure. A joke may be fluent, on-topic, and grammatically correct while still missing the intended effect. At the same time, the problem is not simply that humor is subjective. A prompt can admit many different good jokes, and the observed dataset usually contains only a small subset of them. For this reason, humor is not only a hard generation task, it is also a useful case for studying reasoning and reinforcement learning in non-verifiable domains [Loakman et al., 2025a, Lemmens and De Marez, 2026].

This difficulty has become more important since the arrival of large language models. Earlier computational humor systems were narrow but interpretable: they relied on templates, lexical resources, or specific punning mechanisms [Ritchie, 2001, Amin and Burghardt, 2020]. Modern language models have much broader coverage, yet recent work shows that fluent text is not the same as strong humor competence. Large evaluations still place model systems below strong human performance, and recent studies on puns and joke understanding suggest that models often reproduce the surface form of humor more reliably than they capture its underlying mechanism [Zhang et al., 2024, Xu et al., 2024, Cocchieri et al., 2025, Loakman et al., 2025b]. In other words, humor exposes a familiar weakness of contemporary models: they are good at continuing likely text, but weaker at open-ended search over many plausible and surprising continuations.

One response to this weakness has been to make the reasoning process explicit. Recent humor-generation systems increasingly rely on staged prompting, creative search, or theory-guided decomposition instead of one-shot completion. These approaches are motivated by a simple observation: good jokes are often not

produced by selecting the first likely continuation, but by exploring associations, reframings, contrasts, and punchline formats [Chen et al., 2023, Zhong et al., 2024, Tikhonov and Shtykovskiy, 2024, Kim and Chilton, 2025, Zhang et al., 2025]. The results are encouraging, but they come with a limitation. In most such systems, the reasoning structure is prescribed by hand: the researcher decides how many stages there are, what intermediate representations should be produced, and in what order the model should proceed.

A parallel line of work in post-training suggests a different possibility. In verifiable domains, reinforcement learning has recently been used to elicit extended reasoning behavior directly, rather than only through manual prompting or fixed pipelines [Lightman et al., 2023, DeepSeek-AI, 2025]. Verifier-free methods such as VeriFree and RLPR further suggest that some of this logic may extend beyond mathematics and code, replacing an external verifier with intrinsic probability-based objectives [Zhou et al., 2025, Yu et al., 2025]. For humor, however, these formulations face an immediate mismatch with the task itself. They are built around a single-reference assumption, whereas humor is naturally multi-reference: for a prompt such as "write a joke about a frog" there are many acceptable outputs, and any dataset records only a few.

This thesis is motivated by that mismatch. Its central claim is that humor generation should be formulated not as exact imitation of an observed joke, but as probability allocation over a set of acceptable jokes conditioned on a prompt. That shift matters both theoretically and practically. Theoretically, it changes how verifier-free reinforcement learning must be defined in a creative domain. Practically, it changes what the dataset must look like: instead of isolated prompt-answer pairs, training requires prompt-conditioned reference sets with multiple plausible jokes attached to the same constraint.

The work therefore has two connected parts. The first is methodological and theoretical: it formulates a multi-reference extension of verifier-free reinforcement learning for humor generation and analyzes the consequences of incomplete observed reference sets. The second is implementational: it builds the data pipeline needed to make such an objective operational. In the current repository, this pipeline already covers corpus collection, dense embedding generation, keyword extraction, prompt construction, reference retrieval, candidate generation, and automatic evaluation. The missing stage is the training loop itself. Accordingly,

this thesis should be read as a preliminary study in the exact sense intended by that term: it establishes the formal objective and the software required for the later training experiment.

The data source is a combined corpus of approximately 664k jokes assembled from the Short Jokes and rJokes collections. The full corpus serves as the retrieval pool, while a its keywords are sufficient to construct the multi-reference training dataset. The target experimental setting is narrow and controlled: Qwen3-1.7B and Qwen3-4B models in instruct and thinking modes will be compared against an MRVF-trained thinking variant, with training planned on NVIDIA H200 GPUs using a patched `trl` and `vLLM` for rollout and baseline inference.

Chapter 1

Literature Review

1.1 Computational humor as a research problem

Humor has long been treated as a hard case for natural language processing because successful jokes depend on more than grammatical well-formedness. They also depend on incongruity, shared knowledge, timing, and audience expectations. Early reviews already described humor as one of the more complex targets for artificial intelligence, noting that implemented systems were largely confined to narrow punning mechanisms and that broader humor processing would require substantial world knowledge and reasoning [Ritchie, 2001]. Two later surveys reach a similar conclusion from different angles. Amin and Burghardt [2020] show that computational humor generation has historically traded breadth for control: template- and rule-based systems could work well with identifiable humorous mechanisms, but only within highly restricted domains. More recently, Loakman et al. [2025a] and Lemmens and De Marez [2026] argue that the large-language-model era has broadened coverage while leaving the central problem unresolved, since fluent text is not equivalent to humor that readers actually find funny.

A further difficulty is theoretical pluralism. Humor theory offers several recurring explanatory families—most notably incongruity-based accounts, but also superiority and relief traditions—yet no single framework exhausts the phenomenon. This matters computationally because different joke types rely on different mechanisms. Kao et al. [2016], for instance, formalize incongruity in puns as ambiguity plus distinctiveness and show that these variables predict human funniness judgments. Their work is important not only because it finds an application for a classical humor theory, but also because it illustrates the limits of narrow formalization: a model that explains homophonic puns does not automat-

ically explain narrative jokes, topical jokes, or socially situated humor. Recent theory-driven work in humor detection starts from the same premise, namely that no single comprehensive humor theory is sufficient and that computational models typically learn only fragments of humor [De Marez et al., 2024]. For the present thesis, the consequence is methodological: humor should be treated as a creative task with a multitude of potential mechanisms rather than as a single scalar attribute of communication.

1.2 Evolution of approaches to humor generation

Before the rise of large language models, humor generation systems were usually narrow by design. As summarized by Amin and Burghardt [2020], earlier work often relied on hand-written templates, lexical resources, phonetic transformations, or tightly constrained generation procedures. These systems had an important advantage: they encoded at least one explicit humorous mechanism. Their disadvantage was low adaptability. They produced humor only under carefully chosen linguistic conditions and therefore did not scale well beyond punning, stock joke forms, or limited domains.

Large language models invert that trade-off. They can produce open-ended humorous text over a far broader range of prompts, but fluency does not guarantee robust humor competence. This is visible in recent large-scale evaluations. Zhang et al. [2024] introduce a multimodal cartoon-captioning benchmark built from more than 250 million human ratings over more than 2.2 million captions, and show that current fine-tuning methods remain limited on creative tasks. Even strong systems such as GPT-4 and Claude remain below top human contestants. Humor-specific understanding studies point in the same direction. Xu et al. [2024] show that large language models exhibit what they call lazy pun generation: instead of fusing two meanings into a single compact pun, the model frequently states both related words or meanings separately, which satisfies the task superficially. This finding is important methodologically, because it shows that simple lexical-success metrics can overestimate pun ability. Evaluation must also distinguish structural quality and originality from mechanically assembled or copied wordplay. Cocchieri et al. [2025] construct a contamination-resistant humor benchmark and find that most language models still underperform humans

on pun comprehension, resolution, and generation. Going beyond puns, Loakman et al. [2025b] show that current models, including reasoning models, do not reliably explain all joke types, which highlights how much of the existing literature remains focused on relatively simple joke forms.

Taken together, these studies suggest that current models are better at reproducing the surface form of familiar jokes than at generalizing across humor mechanisms. This distinction is important for the present thesis. A system can be good at producing joke-like text without being good at humor in the stronger sense required by open-ended generation.

1.3 The issues of humor evaluation

Evaluation is a central difficulty in computational humor. Humor is subjective, but the problem is not only subjectivity in the colloquial sense. What counts as a good joke depends on audience, context, delivery, and the dimension being evaluated. Even dataset papers emphasize that humor varies across topic, time, and recipient, making it difficult to identify a single target variable [Weller and Seppi, 2020]. For this reason, recent humor work increasingly avoids treating funniness as a scalar metric.

The clearest example is Sakabe et al. [2025], who evaluate Oogiri responses on six dimensions: novelty, clarity, relevance, intelligence, empathy, and overall funniness. Their results show that current language models generate responses between low- and mid-tier human performance, but they notably lag in empathy. The authors further argue that this deficit helps explain why model-based evaluation does not reproduce human humor assessment well. The significance of this result is twofold. First, it shows that humor quality is not reducible to "is this funny?" Second, it shows that human and model judgments may privilege different qualities.

A complementary perspective comes from Winters and Van der Stockt [2025], who compare human comedians and GPT-4 in a live improvisational setting rather than in an offline text-only benchmark. Their study is valuable because it isolates a stronger comparison point: both humans and the model must respond rapidly to the same audience suggestions. Human jokes are still preferred slightly more often, but the margin is not overwhelming, and the design highlights how

much delivery and situational context affect humor evaluation. This is a useful corrective to benchmark-only thinking. A text that performs well in static judging is not necessarily the text that works best in real comedic interaction.

These findings also clarify the role of LLM-as-a-judge methods. Structured evaluation frameworks such as Hashemi et al. [2024] are useful because they replace one-dimensional scoring with explicit rubrics and calibration. However, that paper also shows that raw LLM rubric judgments do not automatically align well with human ratings. For humor generation, this implies that LLM judges are best treated as measurement instruments with a limited domain of validity, not as transparent replacements for human preference.

1.4 The promise of reasoning for humor generation

One of the most important shifts in recent humor-generation research is the move from one-shot continuation to explicit reasoning or staged creative search. At the prompting level, Chen et al. [2023] model humor generation through step-by-step comedy-writing instructions for GPT-3. Zhong et al. [2024] go further by arguing that standard chain-of-thought prompting is poorly matched to creative tasks, and propose a Leap-of-Thought paradigm for Oogiri-style humor generation that emphasizes associative jumps rather than strictly sequential decomposition.

More explicit generation systems strengthen this line of work. Tikhonov and Shtykovskiy [2024] reconstruct one-liner joke writing as a multistep process and report improvements over zero-shot GPT-4 and other baselines in human evaluation. Kim and Chilton [2025] frame humor generation as the combination of cognitive, social, and creative skills. In their meme-captioning study, skill-augmented systems substantially outperform large language model baselines and approach top-rated human captions for the target audience. In multimodal settings, Zhang et al. [2025] propose a theory-guided multi-stage reasoning framework intended to improve both interpretability and humor quality.

These papers are important because they collectively reject the assumption that humor is best produced by an instant completion. Instead, they model humor generation as a search process over premises, associations, reframings, and audience-sensitive choices, similar to how human experts approach comedy

writing or any creative tasks. An informational gap between the premise and the joke which is only filled out during inference that may be latent. At the same time, they share a limitation that is directly relevant to the present thesis: the structure of the reasoning process is defined rigidly. The number of stages, the type of intermediate representations, and the order of operations are fixed by the researcher. Such systems are therefore strong demonstrations of the usefulness of reasoning, but weaker evidence that a model can learn for itself when and how to reason for humor.

1.5 Standard post-training is a poor fit for humor

The most straightforward route to improving humor generation is supervised fine-tuning on large joke corpora. In practice, however, that objective is a poor match for the structure of the task. Joke datasets contain many repeated setups, familiar wordplay patterns, and conventional punchline forms. As a result, supervised fine-tuning can improve stylistic plausibility without necessarily improving creative generalization. Large humor resources such as rJokes are valuable for scale and topical variety, but they were not constructed as multi-reference generation datasets in which many distinct outputs can simultaneously be correct [Weller and Seppi, 2020].

Recent literature strengthens this concern from two directions. First, humor-specific work shows that even very large preference datasets do not make creative alignment easy. Zhang et al. [2024] explicitly highlight the limitations of RLHF- and DPO-style fine-tuning on creative captioning. Second, post-training work outside humor makes the memorization-generalization distinction sharper. Chu et al. [2025] show that reinforcement learning generalizes more effectively than supervised fine-tuning to unseen textual and visual variants, whereas supervised fine-tuning tends to memorize training patterns and struggle out of distribution. This contrast is especially consequential in humor generation, where a weak prompt can have many good continuations and where memorizing frequent joke shells is not the same as producing an apt and surprising punchline.

Preference optimization has its own difficulties. RLHF assumes that a reward model can learn a stable proxy for the target behavior, but humor judgments are noisy and heterogeneous. Gao et al. [2023] show more generally that

aggressively optimizing an imperfect reward model can harm true performance. Humor-specific evaluation suggests that the risk is amplified in creative domains: Sakabe et al. [2025] show systematic divergence between human and model humor judgments, while Winters and Van der Stockt [2025] show that humor quality depends on aspects of performance that are hard to capture in offline textual reward signals. For these reasons, neither supervised fine-tuning nor standard preference optimization cleanly solves the central problem.

1.6 Reasoning-oriented post-training

The recent success of reasoning-oriented post-training has come mainly from domains with objective verification. Wei et al. [2022] show that intermediate reasoning can substantially change model behavior on tasks that reward multi-step inference. Lightman et al. [2023] extend this logic from prompting to supervision, showing that process supervision outperforms outcome-only supervision on the MATH benchmark. DeepSeek-AI [2025] then demonstrate that reinforcement learning can elicit long-form reasoning without relying on human-authored chains of thought, and the clearest gains are reported in verifiable domains such as mathematics, coding, and related STEM tasks.

This creates the central transfer problem for humor. Humor may also benefit from extended reasoning, yet there is usually no deterministic verifier that can declare a joke correct. Recent verifier-free approaches aimed at domains outside of mathematics and coding attempt to close this gap by replacing the explicit verifiable reward with intrinsic probabilities assigned to reference answers. Zhou et al. [2025] propose VeriFree and show that, under a single-reference assumption, maximizing the conditional probability of the reference answer given the prompt and reasoning trace is equivalent to maximizing an exact-match verifier objective in expectation. Yu et al. [2025] develop a related approach named RLPR where reward is formulated as a mean likelihood over individual tokens to enhance variance stability.

For humor, however, the single-reference assumption is not a minor technical detail, it is the wrong data model. Prompts such as "write a joke about a frog" inherently admit many plausible responses, and any dataset will observe only a small and biased subset of them. In other words, humor is both multi-reference

and only partially observed. This means that verifier-free reasoning methods are conceptually relevant to humor generation, but not directly applicable without reformulating the reward around sets of references rather than single targets.

1.7 Research gap

The literature reviewed above leaves a precise gap. Humor-generation studies increasingly show that staged reasoning, skill decomposition, or theory-guided generation can improve outputs, but these systems mostly rely on rigid inference pipelines [Chen et al., 2023, Zhong et al., 2024, Tikhonov and Shtykovskiy, 2024, Kim and Chilton, 2025, Zhang et al., 2025]. Work on reasoning-oriented reinforcement learning shows that useful reasoning behaviors can be elicited, but the strongest existing formulations assume either objective verifiability or a single reference answer [Lightman et al., 2023, DeepSeek-AI, 2025, Zhou et al., 2025, Yu et al., 2025]. Meanwhile, humor evaluation research shows that funniness is multidimensional, context-sensitive, and only imperfectly captured by model judges [Zhang et al., 2024, Sakabe et al., 2025, Winters and Van der Stockt, 2025, Hashemi et al., 2024].

The present thesis is positioned at the intersection of these findings. It claims that humor benefits from reasoning, but rejects the need to formulate a fixed creative script. It also expands on verifier-free reinforcement-learning, but argues that creative tasks require a multi-reference formulation because multiple outputs can be simultaneously good and the observed reference set is incomplete. The resulting research question is therefore whether verifier-free reasoning objectives can be adapted to partially observed multi-reference humor generation in a way that encourages creative search without collapsing into memorized training samples.

Chapter 2

Methodology

2.1 Research scope and implementation status

The methodology of this preliminary thesis has two layers. The first layer is an implemented data and evaluation pipeline, already present in the repository. The second layer is a proposed training objective, namely multi-reference verifier-free (MRVF) reinforcement learning, which is formulated mathematically but not yet fully implemented in code. This distinction is important for the scope of the present work. The repository already contains pipelines for corpus construction, embedding generation, keyword extraction, reference retrieval, candidate generation, and automatic pairwise evaluation. However, the training stage itself remains future work. Consequently, this chapter distinguishes between what is currently operational in the codebase and what is proposed as the next stage of the project.

At a high level, the implemented system constructs a weakly supervised multi-reference dataset from raw joke datasets. Jokes are first merged into a unified corpus, embedded with a dense retrieval model, and reduced to a small set of keywords with an emb. These keywords are then expanded into n-gram prompt groups, which are then used to retrieve semantically related jokes from the embedding index. The resulting references supports two immediate uses. First, it provides data for baseline generation and automatic evaluation. Second, it defines training and validation sets required by the proposed MRVF training objective.

2.2 Dataset construction

Dataset is merged from two public joke collections: the Short Jokes dataset and the rJokes dataset. In the repository, this stage is implemented in

`src/pipelines/jokes.py`. The pipeline downloads the raw files from pinned source revisions, converts them into parquet format, and preserves source metadata for each joke. For every row, the stored fields are `id`, `text`, `source_name`, `source_filename`, and `source_id`. After preprocessing the two corpora separately, they are concatenated and assigned a new global integer identifier. The final dataset consists of approximately 664 k items

This design is intentionally conservative. The pipeline does not perform aggressive text normalization beyond removing empty entries and standardizing storage format. For humor, such caution is desirable, because seemingly minor textual details, such as punctuation, wording, or line breaks may contribute to comedic effect. Retaining source metadata is also useful for later analysis of domain differences, duplicate detection, and possible source-specific biases.

2.3 Dense embedding index

The next stage embeds each joke in a dense semantic vector space. This is implemented in `src/pipelines/embeddings.py`. The current default configuration uses the model `Qwen3-Embedding-8B` with embedding dimensionality 1024, reduced from the original size of 4096 with Matryoshka Representation Learning. Embeddings are computed asynchronously in batches, written to parquet shards, and stored as pairs (i, e_i), where i is the global joke identifier and $e_i \in \mathbb{R}^{1024}$ is the corresponding embedding.

The role of this stage is twofold. First, the joke embeddings are later used to score candidate keywords by semantic relevance. Second, they define the vector index used for retrieving multi-reference joke sets. In other words, the embedding space is the shared representation on which both keyword extraction and reference retrieval depend.

2.4 Keyword extraction as weak prompt retrieval

Each joke is assumed to have a set of weak prompts that it plausibly answers. For example, "write a joke about a chicken" is considered a viable prompt for "why did the chicken crossed the road" joke. Keyword extraction stage is aimed at constructing weak prompts based on the keywords of jokes. This stage is

implemented in `src/pipelines/keywords.py`.

For each joke y_i, the pipeline first generates a candidate pool of n-grams with a `CountVectorizer` analyzer. Under the default configuration, this analyzer uses unigrams only, removes English stopwords, and keeps at most 64 candidate terms per joke. Let the resulting candidate set be

$$C(y_i) = \{c_{i1}, \ldots, c_{im}\}.$$

Each candidate term is then embedded with the same embedding model used for jokes. If $e(y_i)$ denotes the joke embedding and $e(c)$ denotes the embedding of a candidate keyword, the relevance score is computed by cosine similarity:

$$s(c, y_i) = \frac{e(c)^\top e(y_i)}{\|e(c)\| \, \|e(y_i)\|}.$$

Selecting only the most similar candidates would often yield redundant keywords. To reduce redundancy, the pipeline applies maximal marginal relevance (MMR), choosing multiple keywords while balancing relevance against diversity. Under the default configuration, the diversity coefficient is 0.7 and the number of retained keywords is 3. The output of this stage is therefore a compact keyword representation

$$K(y_i) = \{k_{i1}, k_{i2}, k_{i3}\},$$

possibly with fewer than three items when the candidate pool is small.

Methodologically, these keywords should not be interpreted as full latent prompts. They are weaker objects, n-grams that capture topical or lexical anchors of the joke. This is a pragmatic approximation. It does not solve the joke-to-prompt inversion in a complete sense, but it yields a tractable representation from which multiple prompt options can be constructed.

2.5 Reference construction by keyword grouping and retrieval

2.5.1 Keyword grouping and prompt rendering

The reference construction stage is implemented in `src/pipelines/references.py`. For each joke keyword set $K(y_i)$, the pipeline enumerates all unique keyword combinations whose size lies between `min_keywords` and `max_keywords`. Under the default configuration, these values are 1 and 2, so the pipeline constructs all one-keyword and two-keyword groups. If

$$K(y_i) = \{k_1, k_2, k_3\},$$

then the generated groups are

$$\{k_1\},\ \{k_2\},\ \{k_3\},\ \{k_1, k_2\},\ \{k_1, k_3\},\ \{k_2, k_3\}.$$

Each group is rendered into a natural language prompt using the template

```
Write a joke using the following keyword(s): {keywords}
```

This prompt serves two purposes: it is the textual form later given to generation models, and it is also the object embedded as a query for reference retrieval.

This design creates a controlled but diverse spectrum of prompts. Single keyword prompts are broad, topical and often admit many possible jokes. Two keyword prompts are more constrained and tend to retrieve more precise candidates. The present implementation stops at two keywords because larger groups would quickly become too specific, reducing the number of plausible distinct references per prompt.

2.5.2 Reference retrieval with embedding index

Once prompt strings are formed, the pipeline embeds them as queries and retrieves nearby jokes from a FAISS index built over the full joke-embedding collection. The retrieval index is an inverted file index (IVF). Before indexing and search, vectors are normalized, so that inner product corresponds to cosine similarity. Under the default configuration, the main parameters are:

- embedding model: `Qwen3-Embedding-8B`;
- dimensionality: 1024;
- `faiss_nlist`: 4096;
- `faiss_nprobe`: 32;
- `top_k`: 5;
- `oversample`: 10;
- `min_similarity`: 0.5.

For a rendered prompt x, let $q(x)$ be its normalized embedding and let e_j be the normalized embedding of joke y_j. The retrieval score is then

$$s(x, y_j) = q(x)^\top e_j.$$

The search procedure retrieves more than k candidates, filters out those whose similarity falls below the threshold, and retains the top k remaining items. The corresponding texts are then used as the observed reference set:

$$Y^*(x) = \{y_1, \ldots, y_n\}, \qquad n \leq 5.$$

Note that in practice reference retrieval produces around 4 references per keyword group after deduplication and splitting. The target sample size for training is $10 - 20$k references, hence generating $3 - 5$k keyword groups is enough.

This stage implements the central point of the thesis: a prompt associated with multiple candidate joke references rather than with a single ground truth. Importantly, the retrieved set is not an exhaustively annotated set of all good jokes for x. It is a retrieved approximation of that set. This matters both empirically and theoretically. Empirically, retrieved references may contain noise or omit valid alternatives. Theoretically, the incompleteness of $Y^*(x)$ becomes a core part of the later MRVF analysis rather than a secondary inconvenience.

2.5.3 Deduplication and splitting

After retrieval, the resulting dataset contains rows of the form

$$(x_i, Y^*(x_i), S(x_i)),$$

where x_i is the keyword-based prompt, $Y^*(x_i)$ is the retrieved reference set, and $S(x_i)$ is the vector of retrieval scores. Different source jokes can generate identical keyword groups, so the repository deduplicates the dataset by the `keywords` field, keeps the first occurrence, and reassigns row identifiers.

The deduplicated dataset is then split into train, validation, and test partitions. The current implementation uses a two-stage random split with `test_fraction`=0.1, `validation_fraction`=0.1, and a fixed random seed. Separate parquet shards are written for the full dataset and for each split.

2.6 Candidate generation

This stage supports candidate generation over the reference dataset and is implemented in `src/pipelines/candidates.py`. For a selected model and dataset split, the pipeline iterates over the prompt rows and generates one candidate joke using an OpenAI-compatible Chat Completion API. The generation prompt is the same prompt template used in reference construction:

```
Write a joke using the following keyword(s):  {keywords}
```

Under the default configuration, candidate generation uses temperature 1.0 and allows up to 128 completion tokens. It is important keep the jokes short to simplify evaluation for both LLM-as-a-judge and human settings.

The output schema contains four fields: row identifier, keyword group, model name, and generated text. Candidate files are stored under a per-model, per-split directory structure. This organization is important for later evaluation, because it allows outputs from several models to be aligned by the same underlying identifier and prompt.

Methodologically, this candidate stage should be understood as a benchmarking layer. It provides the artifacts needed to compare baseline models (pretrained base, instruct and thinking versions) and future trained checkpoints.

2.7 Candidate evaluation

2.7.1 Pairwise comparison

Automatic evaluation is implemented in `src/pipelines/evaluation.py`. The pipeline first concatenates candidate datasets from multiple models. It then groups candidates by reference identifier and model name, and constructs pairwise comparisons across all model pairs that produced outputs for the same prompt. Left-right ordering is randomized with a fixed seed in order to reduce position bias.

The evaluation prompt is deliberately simple, it contains the shared joke prompt together with a left and a right candidate. The system instruction tells the annotator to act as an judge of humor quality, choose which joke is funnier for a general audience, avoid ties, and return strict JSON. Under the default configuration, the judge model runs at temperature 0.

This evaluation layer captures only comparative funniness under a shared prompt. It does not yet implement the multidimensional evaluation discussed in the literature review, such as separate rubric-based judgments for prompt adherence, novelty, human-likeness, and funniness, nor does it yet include human annotation. These stages are future work as they depend on the complete training procedure.

2.7.2 Leaderboard aggregation

Pairwise judgments are aggregated into a leaderboard with wins, losses, number of comparisons, win rate, and Bradley-Terry scores. Let $\mathcal{M}$ denote the set of evaluated models. For every judged pair (m_a, m_b), the evaluator records a binary win for one side. The pipeline then estimates Bradley-Terry strengths and reports their logarithms as `bt_score` values.

This aggregation is a sensible choice for humor because relative judgments are often more stable than absolute scores. Asking a judge to decide which of two jokes is funnier is usually easier than asking for a well-calibrated numerical rating. At the same time, this procedure inherits the limitations of the judge model. Since humor judgments by language models can diverge from human judgments, the current leaderboard should be interpreted as a fast and cheap proxy used for

intermediate validation rather than as a definitive measure of humor quality.

2.8 Theoretical framework for MRVF training

The implemented pipeline above constructs prompt-conditioned reference sets, the present section specifies how such sets would enter a reasoning-oriented reinforcement-learning objective. The central claim is that humor generation is not well matched to the single-reference assumptions used in standard verifier-based or verifier-free reasoning RL. A weak prompt may admit many acceptable jokes, and the repository already makes this concrete by retrieving several references for the same keyword-based prompt. The theory below formalizes how to optimize against such set-valued supervision.

2.8.1 Notation

Let $\mathcal{X}$, $\mathcal{Z}$, and $\mathcal{Y}$ denote the spaces of prompts, reasoning traces, and final answers, respectively. For a prompt $x \in \mathcal{X}$,

- $z \in \mathcal{Z}$ denotes a reasoning trace,
- $y \in \mathcal{Y}$ denotes the final generated joke,
- $y^* \in \mathcal{Y}$ denotes a single reference answer,
- $Y^*(x) \subset \mathcal{Y}$ denotes the *observed* set of acceptable references associated with prompt x,
- π_θ denotes the policy, that is, the conditional probability model over text parameterized by θ.

  More explicitly,

$$\pi_\theta(z, y \mid x) = P_\theta(Z = z, Y = y \mid X = x).$$

The policy is factorized as

$$\pi_\theta(z, y \mid x) = \pi_\theta(z \mid x)\, \pi_\theta(y \mid x, z).$$

When the model emits explicit reasoning tokens, z can be identified with the text inside a designated `<think>` segment and y with the final answer that follows it. When reasoning is not explicitly exposed in the output stream, the same factorization can still be read as a conceptual decomposition into a sampled intermediate trajectory z and a final answer conditioned on that trajectory. The advantage of this notation is that it separates two roles of the model: selecting a reasoning path and assigning probability mass to answers given that path.

For notational economy, the optimization objective below is written in terms of the observed set $Y^*(x)$. When discussing incompleteness, it will be useful to imagine a larger latent set $Y_{\text{true}}(x) \supseteq Y^*(x)$ containing all genuinely acceptable jokes for prompt x, even though that full set is never observed in practice.

2.8.2 RL with verifiable rewards

Reinforcement learning optimizes a policy by maximizing expected reward. Given an objective $J(\theta)$, training proceeds by stochastic gradient ascent or, equivalently, by gradient descent on the negative objective. The critical design choice is therefore the reward function: it specifies which generations should receive positive signals.

In verifiable domains, reward can be assigned by an external procedure that does not depend on subjective human judgment. For code, a generated program can be executed against a test suite, for mathematics, a final answer can be parsed and compared against a canonical solution. In such cases the event "the answer is correct" has an meaning external to the language model itself. Recent reasoning models rely heavily on this structure for training [DeepSeek-AI, 2025, Lightman et al., 2023].

Assume first the standard single-reference setting: for each prompt x, there is exactly one verifiable target y^*. Define the reward

$$R(y, y^*) = \mathbb{I}\{y = y^*\},$$

where $\mathbb{I}\{\cdot\}$ is the indicator function. The objective is then

$$J_{\text{RLVR}}(\theta \mid x, y^*) = \mathbb{E}_{z \sim \pi_\theta(z|x)} \mathbb{E}_{y \sim \pi_\theta(y|x,z)} \big[R(y, y^*)\big].$$

Equivalently, in expanded summation form,

$$J_{\mathrm{RLVR}}(\theta \mid x, y^*) = \sum_{z \in \mathcal{Z}} \sum_{y \in \mathcal{Y}} \pi_\theta(z \mid x) \pi_\theta(y \mid x, z)\, R(y, y^*).$$

To optimize this expectation one uses the score-function identity, also known as the REINFORCE identity:

$$\nabla_\theta \, \mathbb{E}_{u \sim p_\theta(u)}[f(u)] = \mathbb{E}_{u \sim p_\theta(u)}\big[f(u) \nabla_\theta \log p_\theta(u)\big].$$

This identity follows directly from the relation $\nabla_\theta p_\theta(u) = p_\theta(u) \nabla_\theta \log p_\theta(u)$. Applying it to the joint distribution of (z, y) yields

$$\begin{aligned}
\nabla_\theta J_{\mathrm{RLVR}} &= \nabla_\theta \sum_{z,y} \pi_\theta(z, y \mid x)\, R(y, y^*) \\
&= \sum_{z,y} R(y, y^*)\, \nabla_\theta \pi_\theta(z, y \mid x) \\
&= \sum_{z,y} \pi_\theta(z, y \mid x)\, R(y, y^*)\, \nabla_\theta \log \pi_\theta(z, y \mid x) \\
&= \mathbb{E}_{z,y}\Big[R(y, y^*)\, \nabla_\theta \log \pi_\theta(z, y \mid x)\Big].
\end{aligned}$$

Using the factorization of the joint policy,

$$\log \pi_\theta(z, y \mid x) = \log \pi_\theta(z \mid x) + \log \pi_\theta(y \mid x, z),$$

hence

$$\nabla_\theta \log \pi_\theta(z, y \mid x) = \nabla_\theta \log \pi_\theta(z \mid x) + \nabla_\theta \log \pi_\theta(y \mid x, z).$$

Therefore

$$\nabla_\theta J_{\mathrm{RLVR}} = \mathbb{E}_{z,y}\Big[R(y, y^*)\big(\nabla_\theta \log \pi_\theta(z \mid x) + \nabla_\theta \log \pi_\theta(y \mid x, z)\big)\Big].$$

This expression has a simple interpretation. A sampled pair (z, y) receives reward only when the verifier declares y correct. When that occurs, the update increases both the probability of the reasoning trace that led to the answer and the probability of the answer itself under that trace

2.8.3 VeriFree

Humor does not provide such an external verifier, and more generally many open-ended generation tasks do not admit a deterministic correctness test. VeriFree addresses this gap by observing that in the single-reference case the expected exact-match reward can be rewritten as a conditional probability [Zhou et al., 2025].

Fix x and z. Then

$$\begin{aligned}\mathbb{E}_{y\sim\pi_\theta(\cdot|x,z)}\big[\mathbb{I}\{y=y^*\}\big] &= \sum_{y\in\mathcal{Y}} \pi_\theta(y \mid x,z)\,\mathbb{I}\{y=y^*\} \\ &= \pi_\theta(y^* \mid x,z).\end{aligned}$$

Substituting this identity back into the RLVR objective gives

$$J_{\mathrm{RLVR}}(\theta \mid x,y^*) = \mathbb{E}_{z\sim\pi_\theta(z|x)}\big[\pi_\theta(y^* \mid x,z)\big].$$

This suggests defining the verifier-free reward

$$r_\theta(x,z,y^*) := \pi_\theta(y^* \mid x,z),$$

and the corresponding objective

$$J_{\mathrm{VF}}(\theta \mid x,y^*) := \mathbb{E}_{z\sim\pi_\theta(z|x)}\big[r_\theta(x,z,y^*)\big].$$

Under the single-reference assumption,

$$J_{\mathrm{VF}}(\theta \mid x,y^*) \equiv J_{\mathrm{RLVR}}(\theta \mid x,y^*)$$

as scalar objectives. The importance of this reformulation is practical as well as conceptual: no explicit decoding of y is required for reward computation, and $\pi_\theta(y^* \mid x,z)$ can be computed by teacher forcing on the reference sequence, which is conveniently parallelizable.

To obtain the policy gradient, write

$$J_{\mathrm{VF}}(\theta \mid x,y^*) = \sum_{z\in\mathcal{Z}} \pi_\theta(z \mid x)\, r_\theta(x,z,y^*).$$

Differentiating by the product rule,

$$\begin{aligned}\nabla_\theta J_{\mathrm{VF}} &= \sum_z \Big[\nabla_\theta \pi_\theta(z \mid x)\, r_\theta(x, z, y^*) + \pi_\theta(z \mid x)\, \nabla_\theta r_\theta(x, z, y^*)\Big] \\ &= \sum_z \pi_\theta(z \mid x)\Big[r_\theta(x, z, y^*)\, \nabla_\theta \log \pi_\theta(z \mid x) + \nabla_\theta r_\theta(x, z, y^*)\Big].\end{aligned}$$

Since $r_\theta(x, z, y^*) = \pi_\theta(y^* \mid x, z)$,

$$\nabla_\theta r_\theta(x, z, y^*) = \pi_\theta(y^* \mid x, z)\, \nabla_\theta \log \pi_\theta(y^* \mid x, z) = r_\theta(x, z, y^*)\, \nabla_\theta \log \pi_\theta(y^* \mid x, z).$$

Hence

$$\nabla_\theta J_{\mathrm{VF}} = \mathbb{E}_{z \sim \pi_\theta(z|x)}\Big[r_\theta(x, z, y^*)\big(\nabla_\theta \log \pi_\theta(z \mid x) + \nabla_\theta \log \pi_\theta(y^* \mid x, z)\big)\Big].$$

This decomposition is informative. The first term,

$$\mathbb{E}_z\big[r_\theta(x, z, y^*)\, \nabla_\theta \log \pi_\theta(z \mid x)\big],$$

is a score-function term over sampled reasoning traces. It assigns higher probability to traces under which the model itself places higher conditional mass on the reference answer. The second term,

$$\mathbb{E}_z\big[\nabla_\theta r_\theta(x, z, y^*)\big],$$

is a direct teacher-forced gradient on the reference sequence. Relative to RLVR, VeriFree removes noise from sampling the final answer y: the estimator still samples z, but not the answer used for reward evaluation. This is precisely why VeriFree is attractive in domains where a verifier is unavailable but a single reference answer exists.

2.8.4 Multi-reference VeriFree

The single-reference assumption is inappropriate for humor. A prompt such as "write a joke using the keywords cat, dog" does not have one correct completion, it has many plausible completions differing in setup, punchline mechanism, tone, and level of absurdity. In fact, the implemented repository is explicitly designed to

construct several references for the same weak prompt. The reward must therefore be defined on a set of acceptable answers rather than on one.

Let $Y^*(x) \subset \mathcal{Y}$ denote the observed set of acceptable references for prompt x. Define the multi-reference verified reward

$$R(y, Y^*) = \mathbb{I}\{y \in Y^*\}.$$

The corresponding set-valued exact-match objective is

$$J_{\mathrm{MR}}(\theta \mid x, Y^*) = \mathbb{E}_{z \sim \pi_\theta(z|x)} \mathbb{E}_{y \sim \pi_\theta(\cdot|x,z)} \big[\mathbb{I}\{y \in Y^*\}\big].$$

For fixed x and z,

$$\begin{aligned} \mathbb{E}_{y \sim \pi_\theta(\cdot|x,z)} \big[\mathbb{I}\{y \in Y^*\}\big] &= \sum_{y \in \mathcal{Y}} \pi_\theta(y \mid x, z)\, \mathbb{I}\{y \in Y^*\} \\ &= \sum_{y \in Y^*} \pi_\theta(y \mid x, z). \end{aligned}$$

This motivates the multi-reference verifier-free reward

$$r_\theta(x, z, Y^*) := \sum_{y \in Y^*} \pi_\theta(y \mid x, z).$$

The quantity $r_\theta(x, z, Y^*)$ lies in $[0, 1]$: it is the total probability mass that the policy assigns, under reasoning trace z, to the observed reference set. Equivalently, it is the probability of the event that the generated answer falls inside that set. The resulting objective is

$$J_{\mathrm{MRVF}}(\theta \mid x, Y^*) = \mathbb{E}_{z \sim \pi_\theta(z|x)} \big[r_\theta(x, z, Y^*)\big].$$

At the scalar level, MRVF plays the same role for a set-valued reward that VeriFree plays for a single exact-match reward. However, the gradient structure is not merely the single-reference formula with y^* replaced by a set symbol. The answer-side derivative changes in an essential way.

Indeed,

$$J_{\mathrm{MRVF}}(\theta \mid x, Y^*) = \sum_z \pi_\theta(z \mid x)\, r_\theta(x, z, Y^*),$$

so

$$\nabla_\theta J_{\mathrm{MRVF}} = \sum_z \Big[\nabla_\theta \pi_\theta(z \mid x)\, r_\theta(x, z, Y^*) + \pi_\theta(z \mid x)\, \nabla_\theta r_\theta(x, z, Y^*)\Big]$$
$$= \mathbb{E}_{z\sim\pi_\theta(z|x)}\Big[r_\theta(x, z, Y^*)\, \nabla_\theta \log \pi_\theta(z \mid x) + \nabla_\theta r_\theta(x, z, Y^*)\Big].$$

Now expand the reference-side derivative:

$$\nabla_\theta r_\theta(x, z, Y^*) = \nabla_\theta \sum_{y\in Y^*} \pi_\theta(y \mid x, z)$$
$$= \sum_{y\in Y^*} \nabla_\theta \pi_\theta(y \mid x, z)$$
$$= \sum_{y\in Y^*} \pi_\theta(y \mid x, z)\, \nabla_\theta \log \pi_\theta(y \mid x, z).$$

Therefore,

$$\nabla_\theta J_{\mathrm{MRVF}} = \mathbb{E}_{z\sim\pi_\theta(z|x)}\Bigg[r_\theta(x, z, Y^*)\, \nabla_\theta \log \pi_\theta(z \mid x) + \sum_{y\in Y^*} \pi_\theta(y \mid x, z)\, \nabla_\theta \log \pi_\theta(y \mid x, z)\Bigg]$$

It is sometimes useful to rewrite the second term in normalized form. Whenever $r_\theta(x, z, Y^*) > 0$, define

$$w_\theta(y \mid x, z, Y^*) := \frac{\pi_\theta(y \mid x, z)}{r_\theta(x, z, Y^*)}\, \mathbb{I}\{y \in Y^*\}.$$

Then $w_\theta(\cdot \mid x, z, Y^*)$ is a probability distribution over the observed reference set, and

$$\nabla_\theta r_\theta(x, z, Y^*) = r_\theta(x, z, Y^*) \sum_{y\in Y^*} w_\theta(y \mid x, z, Y^*) \nabla_\theta \log \pi_\theta(y \mid x, z).$$

Equivalently,

$$\nabla_\theta \log r_\theta(x, z, Y^*) = \sum_{y\in Y^*} w_\theta(y \mid x, z, Y^*) \nabla_\theta \log \pi_\theta(y \mid x, z),$$

and hence

$$\nabla_\theta J_{\mathrm{MRVF}} = \mathbb{E}_{z\sim\pi_\theta(z|x)}\Big[r_\theta(x,z,Y^*)\big(\nabla_\theta \log \pi_\theta(z \mid x) + \nabla_\theta \log r_\theta(x,z,Y^*)\big)\Big].$$

This form clarifies the difference from single-reference VeriFree. In VeriFree, the direct term is the gradient of $\log \pi_\theta(y^* \mid x, z)$ for one reference. In MRVF, the direct term is the gradient of $\log \sum_{y\in Y^*} \pi_\theta(y \mid x, z)$. The update therefore does not privilege one reference a priori, it increases total mass on the observed set, with each reference weighted by the current policy mass assigned to it. If one reference already dominates, the gradient concentrates there. If mass is spread across several references, the gradient reinforces them jointly. This is why MRVF is not simply a notational variant of VeriFree: the optimization geometry changes from a single target string to a log-sum over many targets.

2.8.5 Incomplete observed references

The observed set $Y^*(x)$ is still not the full set of good jokes. Two distinct forms of incompleteness arise.

Structural incompleteness. Even if the repository retrieves several references for a prompt, there may exist many additional jokes in the latent set $Y_{\mathrm{true}}(x)$ that are also valid but unobserved. This is unavoidable in creative domains. A finite dataset cannot enumerate all acceptable punchlines for a weak prompt.

Computational incompleteness. Even within the observed set $Y^*(x)$, computing

$$r_\theta(x,z,Y^*) = \sum_{y\in Y^*} \pi_\theta(y \mid x, z)$$

may become expensive when $|Y^*|$ is large. One may therefore evaluate the sum on a sampled subset $Y \subseteq Y^*$.

Suppose Y is sampled uniformly without replacement from Y^*, and each reference $y \in Y^*$ has inclusion probability

$$\rho = P(y \in Y) = \frac{|Y|}{|Y^*|}.$$

Define the subset reward

$$r_\theta(x, z, Y) := \sum_{y \in Y} \pi_\theta(y \mid x, z) = \sum_{y \in Y^*} \mathbb{I}\{y \in Y\}\, \pi_\theta(y \mid x, z).$$

Taking expectation over the random subset,

$$\begin{aligned}
\mathbb{E}_Y\big[r_\theta(x, z, Y)\big] &= \sum_{y \in Y^*} \mathbb{E}_Y\big[\mathbb{I}\{y \in Y\}\big]\, \pi_\theta(y \mid x, z) \\
&= \sum_{y \in Y^*} P(y \in Y)\, \pi_\theta(y \mid x, z) \\
&= \rho \sum_{y \in Y^*} \pi_\theta(y \mid x, z) \\
&= \rho\, r_\theta(x, z, Y^*).
\end{aligned}$$

Thus the subset reward is proportional in expectation to the full observed-set reward.

This is where GRPO baselining becomes relevant. Suppose that for a fixed prompt x all sampled rewards in a group are multiplied by the same positive factor ρ_x, for example because only a fraction of the observed references is used. If one standardizes rewards within the group,

$$\widetilde{A}_i = \frac{\hat{r}_i - \mu_x}{\sigma_x + \varepsilon}, \qquad \mu_x = \frac{1}{G} \sum_{j=1}^{G} \hat{r}_j,$$

then multiplying every $\hat{r}_i$ by $\rho_x > 0$ multiplies both the numerator and denominator by the same factor, leaving $\widetilde{A}_i$ unchanged. This is the sense in which GRPO group standardization is scale-invariant. It mitigates reward-magnitude shifts in the score-function term, although it does not recover information lost through missing references and does not by itself solve the bias of the direct reference gradient.

Another fundamental issue is the false-negative problem. Optimizing

$$r_\theta(x, z, Y^*) = \sum_{y \in Y^*} \pi_\theta(y \mid x, z)$$

increases probability mass on observed references. Because the conditional distri-

bution over all strings sums to one,

$$\sum_{y \in \mathcal{Y}} \pi_\theta(y \mid x, z) = 1,$$

that increased mass must come from $\mathcal{Y} \setminus Y^*$. Unfortunately, this complement contains not only bad jokes but also unobserved good jokes from $Y_{\text{true}}(x) \setminus Y^*(x)$. In other words, the training signal can inadvertently suppress valid but unseen humorous continuations. This risk is not a defect of the gradient derivation, it is a structural consequence of learning from incomplete supervision in an intrinsically many-solution domain.

To reduce this risk, the proposed optimization includes a Kullback-Leibler penalty relative to a reference policy π_{ref}:

$$\max_\theta \; \mathbb{E}_{x,z}\big[r_\theta(x, z, Y^*(x))\big] - \beta\, \mathbb{E}_x\Big[\mathrm{KL}\big(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x)\big)\Big].$$

Equivalently, one may view the problem in constrained form,

$$\max_\theta \; \mathbb{E}_{x,z}\big[r_\theta(x, z, Y^*(x))\big] \quad \text{s.t.} \quad \mathbb{E}_x\Big[\mathrm{KL}\big(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x)\big)\Big] \le \varepsilon.$$

In the present context the KL term is not merely generic regularization. Its methodological role is to discourage overly aggressive redistribution of probability mass away from the reference model when the observed reference sets are known to be incomplete.

2.8.6 Proposed training loop

The intended training stage samples multiple reasoning traces for the same prompt and uses their rewards to update the reasoning policy. For a prompt x, sample a group

$$z_1, \ldots, z_G \sim \pi_\theta(z \mid x).$$

For each z_i, compute the multi-reference reward

$$\hat{r}_i := r_\theta(x, z_i, Y^*(x)) = \sum_{y \in Y^*(x)} \pi_\theta(y \mid x, z_i).$$

In practice, each $\pi_\theta(y \mid x, z_i)$ is obtained by teacher forcing on the reference sequence y. Since these sequence probabilities are very small, implementation should accumulate token log-probabilities and, where necessary, use stabilization such as $\log \sum \exp$ before converting back to probabilities or normalized weights.

The gradient naturally decomposes into three parts:

$$\nabla_\theta J_{\mathrm{MRVF}} = g_z + g_r + g_{\mathrm{KL}}.$$

Reasoning term. The score-function part over z is estimated from sampled traces and therefore benefits from variance reduction. A clean choice is the leave-one-out baseline

$$b_i = \frac{1}{G-1} \sum_{j \neq i} \hat{r}_j, \qquad A_i = \hat{r}_i - b_i.$$

The corresponding estimator is

$$g_z = \frac{1}{G} \sum_{i=1}^{G} A_i \, \nabla_\theta \log \pi_\theta(z_i \mid x).$$

This is the same logic that motivates RLOO estimators: each trace is compared against the other traces sampled for the same prompt. If desired, one may further normalize the A_i values within the group to reduce prompt-specific reward-scale variation, in the spirit of GRPO. Such normalization improves robustness but is no longer strictly unbiased because the scale estimate itself depends on the sampled group.

Reference term. For the direct answer-side contribution, the correct observed-set gradient is

$$g_r = \frac{1}{G} \sum_{i=1}^{G} \nabla_\theta r_\theta(x, z_i, Y^*(x)) = \frac{1}{G} \sum_{i=1}^{G} \sum_{y \in Y^*(x)} \pi_\theta(y \mid x, z_i) \, \nabla_\theta \log \pi_\theta(y \mid x, z_i).$$

Equivalently, using the normalized within-set weights

$$w_{i,y} := \frac{\pi_\theta(y \mid x, z_i)}{\hat{r}_i} \, \mathbb{I}\{y \in Y^*(x)\},$$

one may write

$$g_r = \frac{1}{G} \sum_{i=1}^{G} \hat{r}_i \sum_{y \in Y^*(x)} w_{i,y} \, \nabla_\theta \log \pi_\theta(y \mid x, z_i),$$

whenever $\hat{r}_i > 0$. This form makes the geometry explicit: each trace receives a total weight $\hat{r}_i$, and within that trace the gradient is distributed across references according to the current policy mass on each reference.

KL term. Finally,

$$g_{\mathrm{KL}} = -\beta \, \nabla_\theta \mathrm{KL}\big(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x)\big).$$

The full update for a minibatch averages these prompt-level contributions over sampled prompts.

The practical importance of the implemented pipeline is now clear. Without prompt-conditioned multi-reference sets $Y^*(x)$, neither $\hat{r}_i$ nor g_r can be computed. The data pipeline is therefore not auxiliary to the theory, it provides the observed structure on which MRVF optimization depends.

2.8.7 Note on baselining

Variance reduction is straightforward only for the score-function term. The reason is that z is genuinely sampled from $\pi_\theta(z \mid x)$. If $b(x, z_{-i})$ is any baseline that does not depend on the sampled variable z_i itself, then

$$\begin{aligned}
\mathbb{E}_{z_i \sim \pi_\theta(\cdot \mid x)}\big[b(x, z_{-i}) \, \nabla_\theta \log \pi_\theta(z_i \mid x)\big] &= b(x, z_{-i}) \sum_{z_i} \pi_\theta(z_i \mid x) \, \nabla_\theta \log \pi_\theta(z_i \mid x) \\
&= b(x, z_{-i}) \sum_{z_i} \nabla_\theta \pi_\theta(z_i \mid x) \\
&= b(x, z_{-i}) \, \nabla_\theta 1 \\
&= 0.
\end{aligned}$$

This is why the leave-one-out baseline is unbiased for the reasoning term.

The reference term is different. It is not an expectation over sampled answers $y \sim \pi_\theta(\cdot \mid x, z)$, it is a direct derivative of the summed reference probability.

Consequently there is no identity of the form

$$\mathbb{E}_y\big[\nabla_\theta \log \pi_\theta(y \mid x, z)\big] = 0$$

available for centering. For the single-reference case, for example,

$$\mathbb{E}_z\big[(r_\theta(x, z, y^*) - b(x))\nabla_\theta \log \pi_\theta(y^* \mid x, z)\big]$$

equals

$$\mathbb{E}_z\big[r_\theta(x, z, y^*)\nabla_\theta \log \pi_\theta(y^* \mid x, z)\big] - b(x)\,\mathbb{E}_z\big[\nabla_\theta \log \pi_\theta(y^* \mid x, z)\big],$$

and the second term is not generally zero. The same issue persists in the multi-reference case with $\nabla_\theta \log r_\theta(x, z, Y^*)$. Therefore baselines that are harmless for the score-function term become biased if naively applied to the direct reference gradient.

This distinction is important for the design of the training algorithm. The reasoning term should use variance-reduction machinery such as leave-one-out baselines or carefully chosen group normalization. The reference term should instead be computed as directly as possible from the observed-set probability, accepting that it is a deterministic but potentially biased proxy for the true reward produced by $Y_{\text{true}}(x)$.

Taken together, these observations explain the overall structure of the proposal. MRVF inherits the key computational advantage of verifier-free learning: it avoids external verifiers and explicit reward sampling over final answers but it must be adapted to the multi-reference, incomplete characteristic of humor. The resulting objective is mathematically tractable, but its limitations are part of the thesis contribution: incompleteness, false negatives, and estimator design are central to the problem.

Chapter 3

Results

3.1 Current results

The implemented pipeline already supports corpus collection, embedding generation, keyword extraction, prompt construction, reference retrieval, candidate generation, and pairwise automatic evaluation. The remaining missing stage is the actual reinforcement-learning update loop. For that reason, this section is narrow: it reports the completed data artifacts and states the concrete experimental setting for the first MRVF training runs.

The corpus-level result is a merged joke collection of approximately 664k items. In the final training setup, this full corpus serves as the retrieval pool: all jokes are embedded and indexed, and references for weak prompts are retrieved from this full embedding space. Keyword extraction, however, only needs to be applied to a sufficiently large subset to hit the target sample size. In our case, $3 - 5$k keyword groups can produce $10 - 20$k references for training, validation and testing.

This is all for the preliminary stage. The important result is that the data and the experimental setting are now finalized: the corpus exists, the reference collection is implemented, the methodology is finalized, and the remaining work is confined to the MRVF training loop.

3.2 Planned training procedure

The repository already implements the full data pipeline in Python using Hugging Face Datasets, parquet artifacts, FAISS-based nearest-neighbor search, and OpenAI-compatible API calls for embedding generation, candidate generation, and automatic evaluation. For the actual training runs, however, the software

stack will change slightly.

The planned implementation will use a patched version of `trl` for optimization and `vLLM` for generation. `trl` provides the trainer-side logic for policy optimization, while `vLLM` will be used both to generate the baseline outputs and to serve rollouts during reinforcement learning. Since MRVF depends on sampling multiple reasoning traces per prompt, rollout throughput matters for efficiency.

The target models are from the Qwen3 family in the 1.7B and 4B size range. For each size, the planned baselines are:

1. **Qwen3 Instruct**, baseline in non-thinking mode;
2. **Qwen3 Thinking**, baseline in thinking mode;
3. **MRVF-trained Qwen3 Thinking**, proposed model;

This setup keeps the comparison interpretable. It isolates three distinct questions: whether ordinary instruction tuning is enough for humor generation, whether native thinking mode already improves humor generation without additional training, and whether MRVF can further improve the thinking model.

All training runs are planned for 1-2 `NVIDIA H200` GPUs on the HSE HPC cluster. This is sufficient for the intended model sizes and allows for parallelization accross instances.

The comparison will use the evaluation pipeline that is already implemented in the repository. Outputs are paired for the same prompt, the left/right order is randomized, the judge is forced to choose exactly one winner, and model scores are aggregated with a Bradley-Terry leaderboard. This is sufficient for the experiment because the goal is to detect whether MRVF produces a measurable shift relative to the instruct and thinking baselines. A larger multidimensional evaluation with rubric-based scoring and human annotation is planned after training is completed.

3.3 Planned ablations

The ablation plan is directly connected to the theory. The most informative ablations are the following:

- **Number of observed references per prompt $|Y^*|$.** Compare smaller and larger reference sets, for example by truncating retrieval to 1, 3, or 5 items.

- **KL coefficient β.** Test how strongly the model must be anchored to the initialization checkpoint in order to fix incompleteness.
- **Group size G.** Vary the number of sampled reasoning traces per prompt in order to assess the variance / throughput trade-off of the MRVF estimator.
- **Model size.** Compare 1.7B versus 4B, and check whether MRVF is robust when applied to smaller models.

References

Mostafa Amin and Manuel Burghardt. A survey on approaches to computational humor generation. In *Proceedings of the 4th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*, pages 29–41, 2020. URL https://aclanthology.org/2020.latechclfl-1.4/.

Yahui Chen, Buxin Shi, and Meng Si. Prompt to GPT-3: Step-by-step thinking instructions for humor generation, 2023. URL https://arxiv.org/abs/2306.13195. arXiv:2306.13195.

Tianyu Chu, Shuhua Zhang, Yu Chen, Ning Ding, and Tao Yu. SFT memorizes, RL generalizes: A comparative study of foundation model post-training, 2025. URL https://openreview.net/forum?id=2P4rYTtGok. OpenReview / arXiv preprint.

Andrea Cocchieri, Lorenzo Ragazzi, Pietro Italiani, Giuseppe Tagliavini, and Gianluca Moro. "what do you call a dog that is incontrovertibly true? dogma": Testing LLM generalization through humor. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 22922–22937, 2025. URL https://doi.org/10.18653/v1/2025.acl-long.1117.

Victor De Marez, Tim Winters, and Ayla Rigouts Terryn. THInC: A theory-driven framework for computational humor detection, 2024. URL https://arxiv.org/abs/2409.01232. arXiv:2409.01232.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025. URL https://doi.org/10.48550/arXiv.2501.12948. arXiv:2501.12948.

Leo Gao, John Schulman, Jacob Hilton, et al. Scaling laws for reward model overoptimization, 2023. URL https://arxiv.org/abs/2210.10760. arXiv:2210.10760.

Hamid Hashemi, Jason Eisner, Corby Rosset, Benjamin Van Durme, and Chris Kedzie. LLM-Rubric: A multidimensional, calibrated approach to automated evaluation of natural language texts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. URL https://aclanthology.org/2024.acl-long.745/.

Justine T. Kao, Roger Levy, and Noah D. Goodman. A computational model of linguistic humor in puns. *Cognitive Science*, 40(5):1270–1285, 2016. URL https://doi.org/10.1111/cogs.12269.

Sangho Kim and Lydia B. Chilton. AI humor generation: Cognitive, social and creative skills for effective humor, 2025. URL https://arxiv.org/abs/2502.07981. arXiv:2502.07981.

Jens Lemmens and Victor De Marez. Computational humor modeling: A survey on the state of the art. *ACM Computing Surveys*, 58(7):177:1–177:37, 2026. URL https://www.uantwerpen.be/en/staff/jens-lemmens/publications/.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023. URL https://arxiv.org/abs/2305.20050. OpenAI / arXiv preprint.

Thomas Loakman, William Thorne, and Chien-Sheng Lin. Who's laughing now? an overview of computational humour generation and explanation, 2025a. URL https://arxiv.org/abs/2509.21175. arXiv:2509.21175.

Thomas Loakman, William Thorne, and Chien-Sheng Lin. Comparing apples to oranges: A dataset & analysis of LLM humour understanding from traditional puns to topical jokes. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 9502–9518, 2025b. URL https://doi.org/10.18653/v1/2025.findings-emnlp.505.

Graeme Ritchie. Current directions in computational humour. *Artificial Intelligence Review*, 16(2):119–135, 2001. URL https://doi.org/10.1023/A:1011610210506.

Ryo Sakabe, Hwiyeon Kim, Tatsuya Hirasawa, and Mamoru Komachi. Assessing the capabilities of LLMs in humor: A multi-dimensional analysis of Oogiri generation and evaluation, 2025. URL https://arxiv.org/abs/2511.09133. arXiv:2511.09133.

Alexey Tikhonov and Pavel Shtykovskiy. Humor mechanics: Advancing humor generation with multistep reasoning, 2024. URL https://arxiv.org/abs/2405.10073. arXiv:2405.10073.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2022. URL https://doi.org/10.48550/arXiv.2201.11903. arXiv:2201.11903.

Orion Weller and Kevin Seppi. The rJokes dataset: A large scale humor collection. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 6136–6141, 2020. URL https://aclanthology.org/2020.lrec-1.753/.

Tim Winters and Sam Van der Stockt. Evaluating humor generation in an improvisational comedy setting. *Computational Linguistics in the Netherlands Journal*, 14:505–523, 2025. URL https://www.clinjournal.org/clinj/article/view/214.

Zhen Xu, Shaojie Yuan, Li Chen, and Diyi Yang. "a good pun is its own reword": Can large language models understand puns? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11766–11782, 2024. URL https://doi.org/10.18653/v1/2024.emnlp-main.657.

Tianyu Yu, Bin Ji, Shun Wang, Shunyu Yao, Zhijian Wang, Ganqu Cui, Luqi Yuan, Ning Ding, Yong Yao, Zhiyuan Liu, Maosong Sun, and Tat-Seng Chua. RLPR: Extrapolating RLVR to general domains without verifiers, 2025. URL https://doi.org/10.48550/arXiv.2506.18254. arXiv:2506.18254.

Jiawei Zhang, Sizhe Luo, Ruijie Zhang, and Qi Su. HUMORCHAIN: Theory-guided multi-stage reasoning for interpretable multimodal humor generation, 2025. URL https://doi.org/10.48550/arXiv.2511.21732. arXiv:2511.21732.

Jie Zhang, Lily Jain, Yichi Guo, Justin Chen, Kexin Linda Zhou, Sreeranjani Suresh, Amanda Wagenmaker, Scott Sievert, Taylor Rogers, Kevin Jamieson, Robert Mankoff, and Robert Nowak. Humor in AI: Massive scale crowd-sourced preferences and benchmarks for cartoon captioning, 2024. URL https://arxiv.org/abs/2406.10522. Advances in Neural Information Processing Systems, Datasets and Benchmarks Track.

Shaoxiong Zhong, Zicheng Huang, Shufan Gao, Wen Wen, Lin Lin, Marinka Zitnik, and Pan Zhou. Let's think outside the box: Exploring leap-of-thought in large language models with creative humor generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13246–13257, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Zhong_Lets_Think_Outside_the_Box_Exploring_Leap-of-Thought_in_Large_Language_CVPR_2024_paper.html.

Xingcheng Zhou, Zihan Liu, Ariel Sims, Han Wang, Tianyu Pang, Chao Li, Li Wang, Min Lin, and Chao Du. Reinforcing general reasoning without verifiers, 2025. URL https://arxiv.org/abs/2505.21493. arXiv:2505.21493.